DATABASE RE-ENGINEERING · CASE STUDY

Applying the PACE Method

Re-engineering a production-planning stored procedure and migrating it from SQL Server to SAP HANA — a structural rebuild focused on robustness, maintainability and native performance.

SQL Server · T-SQL → SAP HANA · SQLScript

Author Raúl Discipline SQL / Data Engineering Asset 1 stored procedure

Method Plan · Analyze · Construct · Evaluate

M0 · M1 · M2 · M3 · M4 · DIF · ENRA · INV · VIV

~900 → ~700
Lines of code — the procedure was rebuilt, not ported.

14 → 6
Temporary tables consolidated into a clear pipeline.

9 → 1
Duplicated stage blocks unified into a single expression.

Null-safe
Every division guarded — no runtime divide-by-zero.

01 Introduction

The source system is a production indicator that answers a single operational question: **how many units are needed at each production stage** to meet the coming weeks of sales demand. It works backwards through the stage chain (M0 → M1 → … → Nursery), weighting every stage by its **productivity (rate)** and its **quality**, where the governing identity is **real factor = quality × rate**. Moving it to SAP HANA could not be done one-to-one: T-SQL and SQLScript differ in syntax, functions and typing. The migration was therefore reframed as a structured re-engineering, guided end to end by the **PACE** method.

02 The PACE walkthrough

Four phases — Plan, Analyze, Construct, Evaluate — took the procedure from a fragile monolith to a native, configurable HANA solution.

P

PHASE 01 · PLAN / PREPARE

Frame the problem and lock the business rules

Inventory the legacy asset and pin down exactly which logic has to be preserved before touching code.

- **Legacy asset:** a single ~900-line T-SQL procedure with 14 temporary tables, deeply nested scalar subqueries and roughly nine near-identical `UNION ALL` blocks.
- **Rules to preserve 1:1:** safety-stock backups per stage (Lab 200%, Rooting 150%, Greenhouse / Nursery 125%), the full stage chain, and the identity `factor = quality × rate`.
- **Target state:** reproduce the results faithfully while making the procedure modular, null-safe and driven by SAP master data instead of hardcoded constants.

A

PHASE 02 · ANALYZE

Map every incompatibility and risk

Separate what actually breaks on HANA from what is merely fragile — and decide what to keep versus redesign.

DIALECT GAPS (NOT IN HANA)

- `IIF()`, `DATEADD / GETDATE / DATEPART`
- Inline `DECLARE @x = (SELECT…)` assignments
- Session `#temp` tables and `COLLATE` clauses
- Implicit type coercion between text and numbers

STRUCTURAL RISKS

- Raw divisions across the stage chain → divide-by-zero & NULL propagation
- Clone-fallback logic copy-pasted nine times
- Stage windows hardcoded per product (`-84`, `-112`, `-56` days)
- Magic numbers scattered through the calculations

C

PHASE 03 · CONSTRUCT

Rebuild in SQLScript — modular, dynamic, null-safe

Rewrite the logic natively and remove the anti-patterns rather than carrying them across.

- **Native SQLScript:** `CASE` for IIF, `ADD_DAYS / CURRENT_DATE / WEEK` for dates, `SELECT … INTO` for variables, explicit `TO_DECIMAL / TO_INT / TO_NVARCHAR` casting for HANA's strict typing.
- **Deduplicated:** the nine fallback blocks collapse into one `COALESCE("FactorFinal","FactorTasa")` inside a single factors table.
- **Consolidated pipeline:** 14 temp tables reduced to 6 clear steps — contamination → activities → quality → rates → factors → output.
- **Data-driven durations:** hardcoded windows externalized to SAP tables (`@TASA_DURACION` , `@COD_ETAPA`); each window is now computed as `DuracionSemanas × 7` .
- **Parameterized by clone:** the whole run is driven by one input via `LIKE :v_clon || '%'` .

● BEFORE · T-SQL — UNGUARDED

```
-- raw division, NULL-prone
DECLARE @VIV FLOAT = (
 SELECT ((@SEMANALODV * SEMANA)
        / FACREAL)
 FROM #FactoresFinales
 WHERE CODETAPA = 9)
```

● AFTER · SQLSCRIPT — NULL-SAFE

```
SELECT IFNULL(
  (v_12SEMANAS * "DuracionEtapa")
  / NULLIF(COALESCE(
      "FactorFinal","FactorTasa"),0),
0)
INTO v_VIV FROM #TEMP_FACTORES
WHERE "CodEtapa" = '09';
```

E

PHASE 04 · EVALUATE

Validate parity and measure the gains

Confirm the numbers still match the original, then quantify the structural improvement.

- **Output parity:** the same staged report — minimum and differentiation units per stage, plus programmed vs. projected production by week, month and year.
- **Simpler:** ~900 → ~700 lines, 14 → 6 temp tables, 9 → 1 fallback expression.
- **More robust:** no divide-by-zero or NULL failures — safe by construction.
- **More maintainable:** stage durations and factors are now configuration, not code; analysts adjust SAP tables instead of editing the procedure.

03 Migration decisions at a glance

The concrete translation of each concern from T-SQL to native SQLScript.

Concern	SQL Server — before	SAP HANA — after
Conditional	`IIF(cond, a, b)`	`CASE WHEN cond THEN a ELSE b END`
Dates	`DATEADD(WW,-52,GETDATE())`	`ADD_DAYS(CURRENT_DATE, -364)`
Variable load	`DECLARE @x = (SELECT …)`	`SELECT … INTO v_x FROM …`
Temp storage	`CREATE TABLE #t — 14 of them`	`LOCAL TEMPORARY TABLE — 6`
Stage fallback	`9 × UNION ALL blocks`	`COALESCE(clone, default)`
Durations	`hardcoded -84 / -112 / -56 days`	`@TASA_DURACION → weeks × 7`
Division safety	`a / b`	`a / NULLIF(b,0) + IFNULL/COALESCE`
Typing	`implicit coercion`	`TO_DECIMAL / TO_INT / TO_NVARCHAR`

Conclusion

PACE turned a risky, impossible-to-port monolith into a native SAP HANA procedure that is materially simpler, safe against runtime failures, and configurable by the business — the same results on a far stronger foundation.

FINAL IMPLEMENTATION

The delivered procedure runs on SAP HANA with dynamic per-clone parameterization, data-driven stage durations sourced from SAP configuration tables, and null-safe calculations end to end — producing consistent weekly production requirements across every stage.